



LAB 3 – PHÁT TRIỂN ỨNG DỤNG WEB JAVASCRIPT VÀ JQUERY

HƯỚNG DẪN NỘP BÀI

Nộp file **MSSV-HoVaTen-Lab3.zip**, bao gồm các file sau:

- MSSV-HoVaTen-Lab3-Report.docx: file báo cáo, đối với từng bài tập, các em cần chụp ảnh phần giao diện và phần code đã thực hiện.
- MSSV-HoVaTen-Lab3-SourceCode.zip: mã nguồn của bài tập.

LÀM QUEN VỚI NGÔN NGỮ JAVASCRIPT, JQUERY

1. Làm quen với ngôn ngữ Javascript

https://www.w3schools.com/js/exercise_js.asp

2. Làm quen với ngôn ngữ JQuery

https://www.w3schools.com/jquery/exercise_jq.asp

PHẦN I – BÀI TẬP CƠ BẢN (8Đ)

NỘI DUNG 1: JAVASCRIPT CƠ BẢN & LOGIC (CÂU 1 - 7)

CÂU 1: THAY ĐỔI NỘI DUNG VÀ MÀU SẮC

- Yêu cầu: Tạo một nút nhấn và một thẻ <h1>. Khi nhấn nút, nội dung thẻ <h1> đổi thành "Chào mừng bạn đã click!" và màu chữ chuyển sang màu đỏ.
- Hướng dẫn: Sử dụng document.getElementById() để truy xuất phần tử. Dùng thuộc tính .innerHTML để đổi chữ và .style.color để đổi màu.
- Mã nguồn mẫu:

```
<h1 id="title">Tiêu đề gốc</h1>
<button onclick="changeContent()">Đổi nội dung</button>

<script>
function changeContent() {
    let element = document.getElementById("title");
    element.innerHTML = "Chào mừng bạn đã click!";
    element.style.color = "red";
}
</script>
```

CÂU 2: KIỂM TRA SỐ CHẴN LẺ

- Yêu cầu: Nhập một số từ input, khi nhấn nút thì thông báo (alert) số đó là chẵn hay lẻ.
- Hướng dẫn: Lấy giá trị từ input bằng .value, chuyển về kiểu số bằng Number(). Sử dụng toán tử chia lấy dư % 2.
- Mã nguồn mẫu:

```
<input type="text" id="numberInput" placeholder="Nhập một số...">
<button onclick="checkEvenOdd()">Kiểm tra</button>

<script>
function checkEvenOdd() {
    let num = Number(document.getElementById("numberInput").value);
    if (document.getElementById("numberInput").value === "" || isNaN(num)) {
        alert("Vui lòng nhập một số hợp lệ!");
    } else {
        let result = (num % 2 === 0) ? "Số chẵn" : "Số lẻ";
        alert(num + " là " + result);
    }
}
</script>
```

CÂU 3: ĐẾM NGƯỢC THỜI GIAN (COUNTDOWN)

- Yêu cầu: Hiển thị số từ 10 và giảm dần về 0 mỗi giây. Khi chạm 0 thì dừng lại.
- Hướng dẫn: Sử dụng setInterval() để thực thi mã sau mỗi 1000ms. Dùng clearInterval() để dừng khi đếm về 0.
- Mã nguồn mẫu:

```

<h2>Thời gian còn lại: <span id="display">10</span> giây</h2>

<script>
let count = 10;
let timer = setInterval(function() {
    count--;
    document.getElementById("display").innerHTML = count;
    if (count <= 0) {
        clearInterval(timer);
        alert("Hết giờ!");
    }
}, 1000);
</script>

```

CÂU 4: HIỆU ỨNG BÓNG ĐÈN (HOVER)

- Yêu cầu: Di chuột vào ảnh bóng đèn thì đèn sáng, di chuột ra thì đèn tắt.
- Hướng dẫn: Gắn sự kiện onmouseover và onmouseout vào thẻ để thay đổi thuộc tính src.
- Mã nguồn mẫu:

```


<p>Rê chuột vào bóng đèn để thắp sáng</p>

<script>
function turnOn() {
    document.getElementById("bulb").src = "bulb_on.png";
}
function turnOff() {
    document.getElementById("bulb").src = "bulb_off.png";
}
</script>

```

CÂU 5: TÍNH TỔNG MẢNG

- Yêu cầu: Cho mảng [5, 10, 15, 20]. Tính tổng và hiển thị khi nhấn nút.
- Hướng dẫn: Sử dụng vòng lặp for hoặc phương thức reduce() để cộng dồn các giá trị trong mảng.
- Mã nguồn mẫu:

```
<p>Mảng hiện tại: [5, 10, 15, 20]</p>
<button onclick="sumArray()">Tính tổng</button>
<h3 id="result"></h3>

<script>
function sumArray() {
  const arr = [5, 10, 15, 20];
  let total = arr.reduce((acc, curr) => acc + curr, 0);
  document.getElementById("result").innerHTML = "Tổng là: " + total;
}
</script>
```

CÂU 6: KIỂM TRA EMAIL HỢP LỆ

- Yêu cầu: Kiểm tra định dạng Email người dùng nhập vào.
- Hướng dẫn: Sử dụng Biểu thức chính quy (Regular Expression) và hàm .test().
- Mã nguồn mẫu:

```
<input type="email" id="emailInput" placeholder="Nhập email của bạn">
<button onclick="validateEmail()">Kiểm tra định dạng</button>

<script>
function validateEmail() {
    let email = document.getElementById("emailInput").value;
    let regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (regex.test(email)) {
        alert("Email hợp lệ!");
    } else {
        alert("Email không đúng định dạng!");
    }
}
</script>
```

CÂU 7: ĐỔI MÀU NỀN NGẪU NHIÊN

- Yêu cầu: Mỗi lần nhấn nút, màu nền trang web thay đổi ngẫu nhiên.
- Hướng dẫn: Tạo mã màu Hex ngẫu nhiên bằng cách lấy ngẫu nhiên các ký tự từ chuỗi 0123456789ABCDEF.
- Mã nguồn mẫu:

```
<button onclick="randomBg()">Đổi màu nền ngẫu nhiên</button>

<script>
function randomBg() {
    let letters = '0123456789ABCDEF';
    let color = '#';
    for (let i = 0; i < 6; i++) {
        color += letters[Math.floor(Math.random() * 16)];
    }
    document.body.style.backgroundColor = color;
}
</script>
```

NỘI DUNG 2: THAO TÁC DOM & BOM (CÂU 8 - 11)

CÂU 8: ĐỒNG HỒ HỆ THỐNG (REAL-TIME)

- Yêu cầu: Hiển thị giờ:phút:giây hiện tại và cập nhật mỗi giây.
- Hướng dẫn: Sử dụng đối tượng new Date() bên trong setInterval.
- Mã nguồn mẫu:

```
<div style="font-size: 24px; font-weight: bold;">
  Giờ hiện tại: <span id="clock">00:00:00</span>
</div>

<script>
setInterval(() => {
  let now = new Date();
  document.getElementById("clock").innerHTML = now.toLocaleTimeString();
}, 1000);
</script>
```

CÂU 9: THÊM VÀ XÓA PHẦN TỬ TRONG DANH SÁCH

- Yêu cầu: Thêm công việc vào danh sách và có nút "Xóa" bên cạnh mỗi mục.
- Hướng dẫn: Sử dụng createElement('li') và appendChild(). Gắn sự kiện .remove() cho nút xóa.
- Mã nguồn mẫu:


```
<input type="text" id="taskInput" placeholder="Nhập công việc...">
<button onclick="addItem()">Thêm</button>
<ul id="taskList"></ul>

<script>
function addItem() {
    let input = document.getElementById("taskInput");
    if (input.value.trim() !== "") {
        let li = document.createElement("li");
        li.innerHTML = input.value + " <button onclick='this.parentElement.remove()'>Xóa</button>";
        document.getElementById("taskList").appendChild(li);
        input.value = ""; // Xóa nội dung input sau khi thêm
    }
}
</script>
```

CÂU 10: KIỂM TRA ĐỘ DÀI MẬT KHẨU

- Yêu cầu: Cảnh báo nếu mật khẩu < 8 ký tự khi đang nhập.
- Hướng dẫn: Sử dụng sự kiện oninput để kiểm tra thuộc tính .length của giá trị input.
- Mã nguồn mẫu:

```
<label>Nhập mật khẩu:</label>
<input type="password" id="passInput" oninput="checkPass()">
<p id="message"></p>

<script>
function checkPass() {
    let pass = document.getElementById("passInput").value;
    let msg = document.getElementById("message");
    if (pass.length === 0) {
        msg.innerHTML = "";
    } else if (pass.length < 8) {
        msg.innerHTML = "Mật khẩu quá ngắn (tối thiểu 8 ký tự)!";
        msg.style.color = "red";
    } else {
        msg.innerHTML = "Mật khẩu hợp lệ";
        msg.style.color = "green";
    }
}
</script>
```

Câu 11: Lưu tên vào LocalStorage

- Hướng dẫn: Sử dụng localStorage.setItem(key, value) để lưu và localStorage.getItem(key) khi trang web load lại (window.onload).
- Mã nguồn mẫu:

```
<input type="text" id="username" placeholder="Nhập tên">
<button onclick="saveName()">Lưu</button>
<script>
    window.onload = () => {
        document.getElementById('username').value = localStorage.getItem('savedName') || "";
    };
    function saveName() {
        let name = document.getElementById('username').value;
        localStorage.setItem('savedName', name);
        alert("Đã lưu!");
    }
</script>
```

NỘI DUNG 3: JQUERY & HIỆU ỨNG (CÂU 12 - 16)

(Lưu ý: Nhúng thư viện jQuery <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script> vào đầu trang)

Câu 12: Ẩn/hiện Menu với slideToggle()

Yêu cầu:

- Thiết kế một giao diện gồm một nút bấm (Button) có nội dung "Danh mục sản phẩm" và một danh sách () chứa ít nhất 3 mục menu.
- Trạng thái ban đầu: Danh sách menu phải được ẩn đi.
- Hành động: Khi người dùng click vào nút bấm, danh sách menu sẽ xổ xuống (nếu đang ẩn) hoặc thu lên (nếu đang hiện) với hiệu ứng trượt mượt mà.

Hướng dẫn giải:

- Bước 1: Nhúng thư viện jQuery từ CDN (ví dụ: Google hoặc cdnjs) vào trong thẻ <head>.
- Bước 2: Sử dụng CSS để đặt thuộc tính display: none; cho thẻ để ẩn menu khi vừa tải trang.
- Bước 3: Sử dụng bộ chọn jQuery \$('#id_nut_bam') để bắt sự kiện .click().
- Bước 4: Bên trong hàm xử lý sự kiện, gọi phương thức .slideToggle() lên bộ chọn của menu. Phương thức này tự động tính

toán trạng thái hiện tại để thực hiện hiệu ứng trượt lên hoặc trượt xuống.

Mã nguồn mẫu:

```
<button id="btnMenu">Toggle Menu</button>
<ul id="menu" style="display:none; background: #eee;">
  <li>Trang chủ</li><li>Sản phẩm</li>
</ul>
<script>
  $('#btnMenu').click(function() { $('#menu').slideToggle(); });
</script>
```

Câu 13: Thêm Class CSS động cho Table

Yêu cầu:

- Tạo một bảng HTML (<table>) hiển thị danh sách sinh viên gồm các cột: STT, Họ Tên, Lớp. Bảng có ít nhất 4 dòng dữ liệu.
- Viết mã CSS cho một class có tên là .highlight (ví dụ: đổi màu nền thành vàng, chữ đậm, con trỏ chuột biến thành bàn tay).
- Hành động: Sử dụng jQuery để khi người dùng click vào một dòng (<tr>) bất kỳ, dòng đó sẽ được áp dụng class .highlight. Nếu click lại lần nữa vào chính dòng đó, class sẽ được gỡ bỏ (hiệu ứng bật/tắt).

Hướng dẫn giải:

- Bước 1: Chuẩn bị cấu trúc bảng HTML chuẩn với các thẻ <thead>, <tbody> và <tr>.
- Bước 2: Định nghĩa class .highlight trong thẻ <style>. Đây là bước quan trọng để jQuery có thể "gọi" các định dạng này ra.
- Bước 3: Sử dụng bộ chọn jQuery \$('tbody tr') để nhắm mục tiêu vào các dòng dữ liệu trong thân bảng.
- Bước 4: Sử dụng phương thức .toggleClass('highlight'). Phương thức này cực kỳ tiện lợi vì nó tự động kiểm tra: nếu phần tử chưa có class thì thêm vào, nếu có rồi thì xóa đi.

Mã nguồn:

```
<style>.highlight { background-color: yellow; }</style>
<table border="1">
  <tr><td>Dòng 1</td></tr><tr><td>Dòng 2</td></tr>
</table>
<script>
  $('tr').click(function() { $(this).toggleClass('highlight'); });
</script>
```

Câu 14: Accordion đơn giản

Yêu cầu:

- Thiết kế một cấu trúc gồm 3 cụm nội dung. Mỗi cụm bao gồm: Một tiêu đề (<h3>) và một khối nội dung tương ứng (<div>).
- Trạng thái ban đầu: Chỉ hiển thị nội dung của mục đầu tiên, các mục còn lại phải được ẩn đi.
- Hành động: Khi người dùng click vào một tiêu đề:
 - Nếu nội dung của tiêu đề đó đang đóng, nó sẽ trượt xuống (slideDown) để hiển thị.
 - Quan trọng: Đồng thời, tất cả các nội dung của các tiêu đề khác phải tự động trượt lên (slideUp) để đóng lại (chỉ cho phép mở duy nhất một mục tại một thời điểm).

Hướng dẫn giải:

- Bước 1: Xây dựng cấu trúc HTML phân lớp. Thường là một thẻ bao ngoài (container) chứa các cặp tiêu đề và nội dung nằm kế tiếp nhau.
- Bước 2: Sử dụng CSS để ẩn các khối nội dung (display: none). Có thể dùng class hoặc trực tiếp ẩn bằng jQuery khi trang vừa load.
- Bước 3: Sử dụng sự kiện .click() của jQuery cho các thẻ tiêu đề h3.
- Bước 4: Trong hàm xử lý, sử dụng \$(this).next() để xác định khối nội dung ngay sau tiêu đề vừa click. Dùng .slideUp() cho toàn bộ các khối nội dung khác và .slideDown() cho khối hiện tại.

Mã nguồn mẫu:

```
<div class="accordion">
  <h3>Tiêu đề 1</h3><div class="content">Nội dung 1</div>
  <h3>Tiêu đề 2</h3><div class="content" style="display:none">Nội dung 2</div>
</div>
<script>
  $('<div class="content">Nội dung 1</div>'.accordion h3').click(function() {
    $('<div class="content">Nội dung 1</div>'.content').slideUp();
    $(this).next('<div class="content">Nội dung 2</div>'.content').slideDown();
  });
</script>
```

Câu 15: Phóng to hình ảnh (LightBox)

Yêu cầu:

- Giao diện chính: Hiển thị một danh sách ít nhất 3 hình ảnh thu nhỏ (thumbnail).
- Giao diện lớp phủ (Overlay): Tạo một lớp nền mờ màu đen bao phủ toàn bộ màn hình (mặc định ẩn). Ở giữa lớp nền này là một khung hiển thị hình ảnh lớn.
- Hành động: * Khi người dùng click vào một hình ảnh thu nhỏ, lớp phủ sẽ hiện lên với hiệu ứng fadeIn.
 - Hình ảnh lớn trong lớp phủ phải khớp với hình ảnh mà người dùng vừa click.
 - Khi người dùng click vào bất kỳ đâu trên lớp phủ (hoặc một nút đóng), lớp phủ sẽ biến mất (fadeOut).

Hướng dẫn giải:

- Bước 1 (HTML): Tạo các thẻ cho ảnh thu nhỏ. Tạo một thẻ <div> có id="lightbox-overlay" chứa một thẻ trống bên trong để làm khung hiển thị ảnh lớn.
- Bước 2 (CSS): Thiết lập cho #lightbox-overlay có position: fixed, top: 0, left: 0, width: 100%, height: 100%, background: rgba(0,0,0,0.8) và quan trọng nhất là display: none.
- Bước 3 (jQuery - Mở): Bắt sự kiện click vào các ảnh thu nhỏ. Lấy giá trị thuộc tính src của ảnh vừa click bằng .attr('src'), sau

đó gán giá trị đó vào thẻ bên trong lớp phủ. Gọi .fadeIn() để hiển thị lớp phủ.

- Bước 4 (jQuery - Đóng): Bắt sự kiện click vào chính lớp phủ và gọi .fadeOut().

Mã nguồn mẫu:

```
<style>
  #overlay { position:fixed; top:0; left:0; width:100%; height:100%; background:rgba(0,0,0,0.8); display:none; text-align:center; }
  #overlay img { margin-top: 50px; max-width: 80%; }
</style>

<div id="overlay"><img src=""></div>
<script>
  $('.thumb').click(function() {
    $('#overlay img').attr('src', $(this).attr('src'));
    $('#overlay').fadeIn();
  });
  $('#overlay').click(function() { $(this).fadeOut(); });
</script>
```

Câu 16: Animate di chuyển phần tử

Yêu cầu: Tạo một khối vuông màu đỏ. Sử dụng hàm .animate() trong jQuery để khi nhấn nút, khối vuông này di chuyển sang phải 300px và xoay tròn.

Mã nguồn mẫu:

```
<div id="box" style="width:50px; height:50px; background:red; position:relative;"></div>
<button id="go">Chạy!</button>
<script>
  $('#go').click(function() {
    $('#box').animate({ left: '300px', opacity: '0.5' }, 1000);
  });
</script>
```

NỘI DUNG 4: AJAX & KẾT NỐI API (CÂU 17 - 20)

Câu 17: Load danh sách User (GET AJAX)

Yêu cầu:

- Giao diện: Tạo một nút bấm "Tải danh sách người dùng" và một bảng HTML (<table>) trống có sẵn dòng tiêu đề (ID, Name, Email, Company).
- Hành động: Khi nhấn nút, sử dụng jQuery Ajax để gửi yêu cầu GET đến API: <https://jsonplaceholder.typicode.com/users>.
- Xử lý dữ liệu: * Duyệt qua mảng dữ liệu trả về từ API.
 - Với mỗi đối tượng người dùng, hãy tạo một dòng (<tr>) mới.
 - Lưu ý: Thông tin Company trong API là một object, bạn cần truy cập vào thuộc tính name của nó (ví dụ: `user.company.name`).
 - Đổ toàn bộ các dòng này vào phần thân bảng (<tbody>).

Hướng dẫn giải:

- Bước 1 (HTML): Chuẩn bị khung bảng với các cột tương ứng. Đặt id cho thẻ <tbody> để dễ dàng chèn dữ liệu sau này.
- Bước 2 (jQuery GET): Sử dụng hàm \$.get(url, function(data) { ... }) hoặc \$.ajax({ type: 'GET', ... }).
- Bước 3 (Vòng lặp): Sử dụng hàm .forEach() của Javascript hoặc \$.each() của jQuery để lặp qua danh sách data.
- Bước 4 (Dom Injection): Tạo một chuỗi HTML chứa các thẻ <td> và dùng phương thức .html() hoặc .append() của jQuery để hiển thị lên màn hình.

Mã nguồn mẫu:


```

<button id="loadUser">Tải danh sách</button>
<table border="1" id="userTable">
  <thead><tr><th>ID</th><th>Name</th><th>Email</th></tr></thead>
  <tbody></tbody>
</table>
<script>
  $('#loadUser').click(function() {
    $.get('https://jsonplaceholder.typicode.com/users', function(data) {
      let rows = '';
      data.forEach(user => {
        rows += `<tr><td>${user.id}</td><td>${user.name}</td><td>${user.email}</td></tr>`;
      });
      $('#userTable tbody').html(rows);
    });
  });
</script>

```

Câu 18: Gửi dữ liệu đăng ký (POST)

Yêu cầu:

- Giao diện: * Tạo một Form đăng ký gồm 2 ô nhập liệu: Username và Password.
 - Một nút bấm "Đăng ký".
 - Một thẻ bên cạnh hoặc bên dưới nút bấm để hiển thị trạng thái phản hồi từ Server.
 - Hành động: Khi người dùng nhấn nút "Đăng ký", sử dụng phương thức \$.post() của jQuery để gửi dữ liệu lên API giả lập: <https://jsonplaceholder.typicode.com/posts>.
 - Xử lý kết quả: * Nếu gửi thành công (Server trả về mã 201), hãy hiển thị thông báo: "Đăng ký thành công! ID tài khoản: [ID từ API]" vào thẻ .
 - Đổi màu chữ của thông báo thành màu xanh lá cây.
- Hướng dẫn giải:**
- Bước 1 (HTML): Tạo form với các id tương ứng cho từng input để dễ dàng lấy giá trị.
 - Bước 2 (Lấy dữ liệu): Khi nhấn nút, dùng .val() để lấy giá trị từ ô Username và Password.

- Bước 3 (jQuery POST): Sử dụng hàm \$.post(url, data, callback).
 - url: Địa chỉ API.
 - data: Một Object chứa các thông tin cần gửi (ví dụ: { user: username, pass: password }).
 - callback: Hàm xử lý sau khi Server phản hồi.
- Bước 4 (Hiển thị): Dùng .text() và .css() để cập nhật thông báo vào thẻ .

Mã nguồn mẫu:

```
<input id="user" placeholder="User"> <input id="pass" type="password">
<button id="signup">Đăng ký</button> <span id="regStatus"></span>
<script>
    $('#signup').click(function() {
        $.post('https://jsonplaceholder.typicode.com/posts', { u: $('#user').val(), p: $('#pass').val() }, function(res) {
            $('#regStatus').text("Thành công! ID: " + res.id).css('color', 'green');
        });
    });
</script>
```

Câu 19: Thanh tìm kiếm trực tiếp (Live Search)

Yêu cầu:

- Giao diện: * Một ô nhập liệu (<input type="text">) làm thanh tìm kiếm.
 - Một danh sách () chứa ít nhất 5-7 mục dữ liệu mẫu (ví dụ: tên các loại điện thoại như iPhone, Samsung, Oppo, Xiaomi, Realme, Nokia, Vivo).
- Hành động: * Khi người dùng gõ bất kỳ ký tự nào vào ô tìm kiếm (sự kiện keyup), danh sách bên dưới sẽ tự động lọc.
 - Chỉ hiển thị những mục có chứa ký tự khớp với nội dung đã nhập (không phân biệt chữ hoa, chữ thường).
 - Những mục không khớp sẽ bị ẩn đi.

Hướng dẫn giải:

- Bước 1 (HTML): Tạo input và danh sách với các thẻ . Đặt ID cho input để dễ truy xuất.
- Bước 2 (Sự kiện): Sử dụng sự kiện .on('keyup', function() { ... }) trong jQuery để bắt khoảnh khắc người dùng buông phím.

- Bước 3 (Lấy giá trị): Dùng `.val()` để lấy chuỗi đang nhập và dùng `.toLowerCase()` để chuyển về chữ thường (giúp tìm kiếm chính xác hơn).
- Bước 4 (Lọc dữ liệu): * Duyệt qua tất cả các thẻ `<li>` bằng hàm `.filter()`.
 - Kiểm tra xem văn bản trong thẻ `<li>` (dùng `.text()`) có chứa chuỗi đang tìm hay không bằng hàm `.indexOf()`.
 - Sử dụng hàm `.toggle()` để ẩn/hiện dựa trên kết quả kiểm tra (nếu `indexOf > -1` thì hiện, ngược lại thì ẩn).

Mã nguồn mẫu:

```
<input type="text" id="search" placeholder="Tìm tên...">
<ul id="pList">
  <li>Apple</li><li>Samsung</li><li>Oppo</li><li>Xiaomi</li>
</ul>
<script>
  $('#search').on('keyup', function() {
    let value = $(this).val().toLowerCase();
    $('#pList li').filter(function() {
      $(this).toggle($(this).text().toLowerCase().indexOf(value) > -1);
    });
  });
</script>
```

Câu 20: Tải thêm dữ liệu (Load More)

Yêu cầu:

- Giao diện: * Một vùng chứa danh sách bài viết (`<div id="post-list">`).
 - Một nút bấm "Xem thêm bài viết" nằm phía dưới danh sách.
- Hành động: * Khi trang vừa tải xong, tự động gọi API để hiển thị 3 bài viết đầu tiên.
 - Khi người dùng nhấn nút "Xem thêm", hệ thống sẽ gọi API để lấy 3 bài viết tiếp theo và nối tiếp (append) vào cuối

danh sách hiện tại.

- API sử dụng: <https://jsonplaceholder.typicode.com/posts>.
- Kỹ thuật: Sử dụng các tham số `_start` (vị trí bắt đầu) và `_limit` (số lượng bản ghi) của API để phân trang dữ liệu.

Hướng dẫn giải:

- Bước 1 (Biến điều khiển): Khai báo một biến toàn cục (ví dụ: `let start = 0;`) để theo dõi vị trí bắt đầu lấy dữ liệu. Mỗi lần tải thành công, ta cộng biến này thêm 3.
- Bước 2 (Hàm tải dữ liệu): Viết một hàm `loadPosts()`. Trong hàm này:
 - Gọi `$.get()` với URL có tham số động: `.../posts?_start=${start}&_limit=3`.
 - Sử dụng vòng lặp để tạo cấu trúc HTML cho từng bài viết (thường gồm Tiêu đề và Nội dung).
 - Dùng phương thức `.append()` của jQuery để đưa dữ liệu mới vào cuối vùng chứa.
- Bước 3 (Sự kiện): Gọi hàm `loadPosts()` lần đầu khi `$(document).ready()`. Sau đó, gắn hàm này vào sự kiện `.click()` của nút "Xem thêm".

Mã nguồn mẫu:

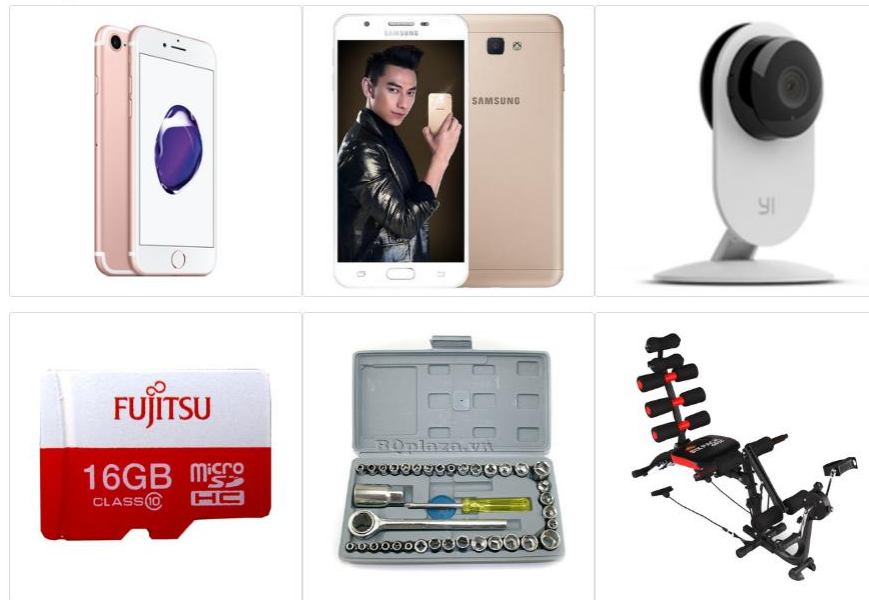
```
<div id="postContainer"></div>
<button id="loadMore">Xem thêm</button>
<script>
  let start = 0;
  function loadPosts() {
    $.get(`https://jsonplaceholder.typicode.com/posts? start=${start}&_limit=3`, function(data) {
      data.forEach(post => {
        $('#postContainer').append('<div><h4>${post.title}</h4></div><hr>');
      });
      start += 3;
    });
  }
  loadPosts(); // Load 3 cái đầu
  $('#loadMore').click(loadPosts);
</script>
```

PHẦN 2. BÀI TẬP NÂNG CAO (2Đ)

Hướng dẫn:

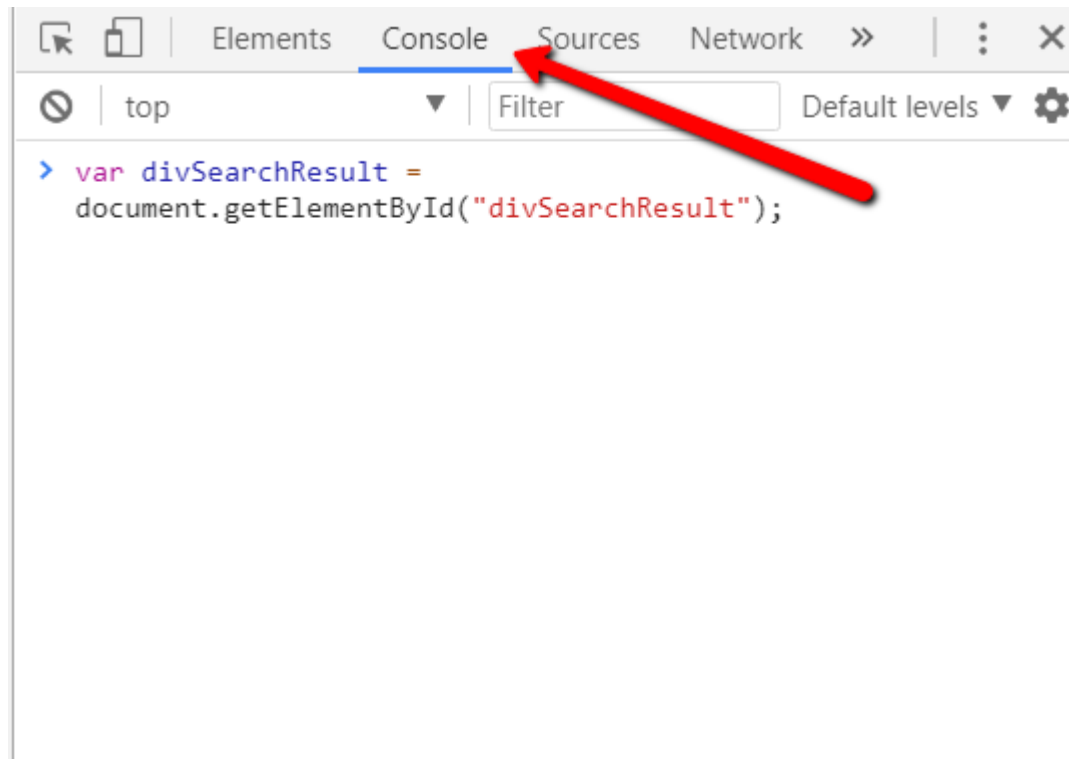
- Giải nén **lab3.zip** > **Source Code** > **BT Nang Cao** ta sẽ được một template (chuẩn bị bởi *Phong Phan*), đã tích hợp sẵn một số framework sau:
 - Bootstrap
 - Google Fonts (đã chọn sẵn font Roboto)
 - FontAwesome
- Các em có thể giải bài tập sử dụng Javascript hoặc JQuery đều được.

Kết quả tìm kiếm



Yêu cầu bài tập:

1.1 Nhấp nút phải chuột Chrome, bấm chọn Console, tiến hành code 1 vài đoạn Javascript cơ bản trong Console, cho chạy thử



1.2. Chạy đoạn javascript sau:

```
document.getElementById("divSearchResult").innerHTML = "YOU CLICKED ME!";
```

Một số hàm khác cùng chức năng tìm kiếm HTML element:

Method	Description
<code>document.getElementById(<i>id</i>)</code>	Find an element by element id
<code>document.getElementsByTagName(<i>name</i>)</code>	Find elements by tag name
<code>document.getElementsByClassName(<i>name</i>)</code>	Find elements by class name

Sau khi tìm được HTML element, các em dùng các hàm bên dưới để thay đổi nội dung HTML element:

Method	Description
<code><i>element</i>.innerHTML = <i>new html content</i></code>	Change the inner HTML of an element
<code><i>element</i>.attribute = <i>new value</i></code>	Change the attribute value of an HTML element
<code><i>element</i>.setAttribute(<i>attribute</i>, <i>value</i>)</code>	Change the attribute value of an HTML element
<code><i>element</i>.style.<i>property</i> = <i>new style</i></code>	Change the style of an HTML element

1.3. Chạy đoạn javascript sau:

```
var x = 3;
var y = 7;
var z = 9;
var result = x+y+z;
document.getElementById("divSearchResult").innerHTML = result;
```

1.4. Gán sự kiện onClick cho nút Xem thêm, để thêm vào một ảnh như sau:

```
var btnSeeMore = document.getElementById("btnSeeMore");
btnSeeMore.onclick = function() {
    myFunction()
};

function myFunction() {
    document.getElementById("divSearchResult").innerHTML
    += '';
}
```

1.5. Tập sử dụng các sự kiện và tiến hành thay đổi nội dung DOM element

Cho một mảng chứa link hình ảnh của tất cả sản phẩm.

```
var products = [
    "images/1.jpg",
    "images/2.jpg",
    "images/3.jpg",
    "images/4.jpg",
    "images/5.jpg",
    "images/6.jpg",
    "images/7.jpg",
    "images/8.jpg",
    "images/9.jpg",
    "images/10.jpg",
    "images/11.jpg",
    "images/12.jpg",
    "images/13.jpg",
    "images/14.jpg",
    "images/15.jpg",
    "images/16.jpg",
    "images/17.jpg",
```



```
"images/18.jpg",  
"images/19.jpg",  
"images/20.jpg",  
"images/21.jpg",  
"images/22.jpg",  
"images/23.jpg",  
"images/24.jpg"
```

```
];
```

```
<!-- Place your code at here! -->
```

```
<div id="main">
```

```
  <div class="container">
```

```
    <h2>Kết quả tìm kiếm</h2>
```

```
    <!-- Grid system -->
```

```
    <div class="row">
```

```
      
```

```
      
```

```
      
```

```
      
```

```
      
```

```
      
```

```
    </div>
```

```
  </div>
```

```
</div>
```

Xóa hết đoạn
này

1.6 Sử dụng sự kiện **Document.OnLoad** để hiển thị tất cả **24 sản phẩm** ở ngay vị trí vừa xóa. Layout 3-1-1 (Dùng vòng lặp)

```
//Add onload event to window object
window.onload = function () {
    //Clear divSearchResult
    var divSearchResult = document.getElementById('divSearchResult');
    divSearchResult.innerHTML = "";

    //Load 24 images
    for (i = 1; i <= 24; i++) {
        divSearchResult.innerHTML += '';
    }
}
```

1.7 Gán sự kiện **onClick** vào nút **Search**, rồi tiến hành lập trình cho nút search như sau (Dùng If Else)

- Nhập vào từ khóa “điện thoại”, bấm nút Search, hiển thị tất cả điện thoại ra.
- Nhập vào từ khóa “nôi”, bấm nút Search, hiển thị tất cả nôi ra.
- Nhập vào từ khóa “sữa rửa mặt”, bấm nút Search, hiển thị tất cả nôi ra.

```
//Implement search function
document.getElementById("btnSearch").onclick = function()
{
    //Get divSearchResult
    var divSearchResult = document.getElementById('divSearchResult');

    //Get keyword entered by user
    var keyword = document.getElementById("txtKeyword").value;
    if(keyword == "")
    {
        alert("Please enter keyword before search !!")
        return;
    }

    //Clear divSearchResult
    divSearchResult.innerHTML = "";
}
```

```

if(keyword == "dien thoai"){
    divSearchResult.innerHTML += '';
    divSearchResult.innerHTML += '';
}
else if(keyword == "noi"){
    divSearchResult.innerHTML += '';
    divSearchResult.innerHTML += '';
}
else if(keyword == "sua rua mat"){
    divSearchResult.innerHTML += '';
    divSearchResult.innerHTML += '';
}
else
{
    divSearchResult.innerHTML = "No product found !";
}

```

1.8. Gán sự kiện khi Scroll tới cuối trang thì sẽ thêm vào 4 tấm ảnh ở cuối divSearchResult.

-----GOOD LUCK-----